LAPORAN PRAKTIKUM

PEMROGRAMAN BERORIENTASI OBJEK

ENKAPSULASI (ENCAPSULATION)



Identitas Mahasiswa

Nama	: Diandras Rio Triasmoro
NIM	: 2530521062
Kelas	: 3 TI C

PROGRAM STUDI TEKNIK INFORMATIKA
FAKULTAS SAINS DAN TEKNOLOGI
UNIVERSITAS MUHAMMADIYAH SUKABUMI
2026

1. **Tujuan dan Ringkasan Konsep**

   Tujuan Praktikum Praktikum ini bertujuan agar mahasiswa dapat memahami dan menerapkan konsep enkapsulasi dalam Pemrograman Berorientasi Objek (PBO). Mahasiswa diharapkan mampu membedakan hak akses (*access modifier*) seperti public, private, dan protected, serta mampu melakukan *refactoring* aplikasi CRUD dengan memanfaatkan *getter* dan *setter* untuk menjaga aturan bisnis dari suatu objek.

   Enkapsulasi adalah proses membungkus data beserta perilaku yang mengelola data tersebut ke dalam satu kesatuan class . Tujuannya adalah untuk melindungi state dari suatu objek agar tidak bisa diubah secara sembarangan dari luar class. Pengendalian ini dilakukan dengan mengubah properti menjadi private atau protected, lalu memberikan jalur resmi yang terkontrol berupa method setter untuk mengubah data dan getter untuk membaca data.

2. **Cuplikan Kode dan Penjelasan Perubahan (Refactoring)**

   Sebelum Refactoring :

   Properti pada class dibiarkan bersifat public, sehingga dari luar class data bisa diakses dan dimanipulasi secara langsung tanpa adanya pemeriksaan atau validasi.

```
public int $semester;

$m->semester = -7;
$this->data[$m->nim] = $m;
```

   Sesudah Refactoring :

   Properti diubah menjadi private, menutup akses manipulasi langsung. Perubahan nilai harus melewati gerbang validasi di dalam constructor atau method *setter*.

```
private int $semester;
public function setSemester(int $semester): void {
    if ($semester < 1 || $semester > 14) throw new
InvalidArgumentException("Semester harus 1-14");
    $this->semester = $semester;
}
public function getSemester(): int { return $this->semester; }

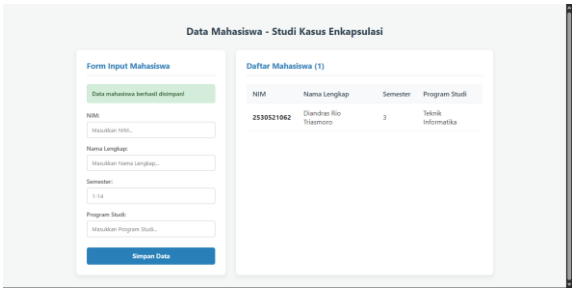
$this->data[$m->getNim()] = $m;
```

   Penjelasan Perubahan :

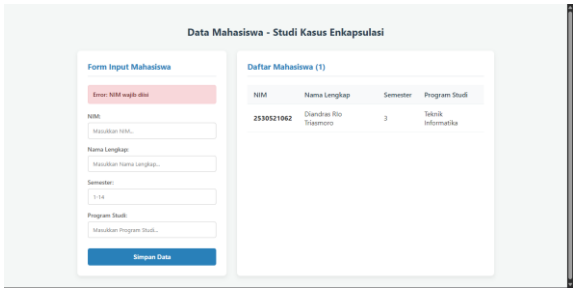
   Perubahan akses dari public ke private memaksa sistem dari luar class untuk menggunakan getter seperti getNim() dan getSemester(). Cara ini memastikan bahwa

identitas mahasiswa dan aturannya tidak dirusak secara langsung, karena interaksi dengan objek selalu melewati gerbang resmi yang sudah dibuat.

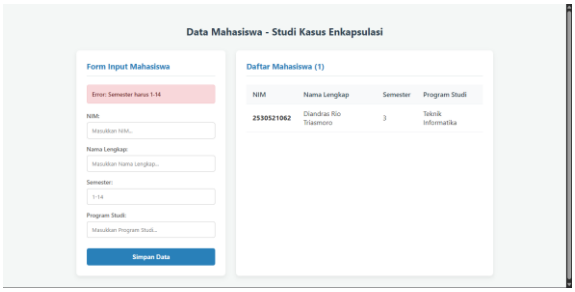
3. Screenshot aplikasi CRUD dan tabel hasil pengujian



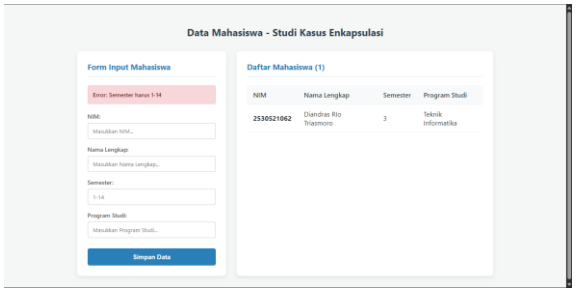
Gambar 1 Data normal



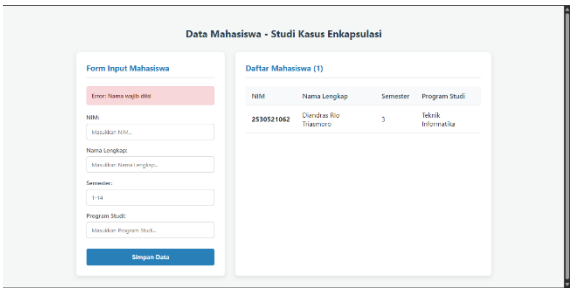
Gambar 2 Nim kosong



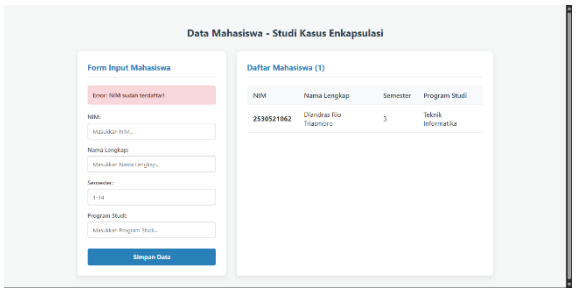
Gambar 4 Semester -1



Gambar 4 Semester 15



Gambar 5 Nama kosong



Gambar 6 Duplikat NIM

No	Kasus Uji	Input/Aksi	Hasil yang Diharapakan
1	Normal	Input mahasiswa sesuai intruksi	Object berhasil dibuat, tersimpan di Session, dan tampil di tabel.
2	Nim kosong	Form NIM dikosongkan lalu simpan	Ditolak dengan pesan validasi "Error: NIM wajib diisi!".
3	Semester -1	Input semester dengan angka -1	Ditolak oleh setter pesan: "Error: Semester harus 1-14".
4	Semester 15	Input semester dengan angka 15	Ditolak oleh setter pesan: "Error: Semester harus 1-14".
5	Nama Kosong	Form Nama berisi spasi kosong	Ditolak oleh setter pesan: "Error: Nama wajib diisi!".
6	Duplikat NIM	Memasukkan NIM yang sudah ada	Ditolak oleh repository pesan: "Error: NIM sudah terdaftar!".

4. Analisis Kasus

Analisis Error Akses Private :

Ketika kode pada file index.php mencoba memanggil langsung variabel $m->semester, program akan berhenti dan memunculkan Fatal error: Cannot access private property. Error ini membuktikan bahwa enkapsulasi berhasil memblokir akses paksa dari luar, sehingga data hanya bisa dibaca dengan cara memanggil method resmi $m->getSemester().

Analisis Validasi Input :

Saat pengguna secara sengaja memasukkan angka semester 15, method setSemester() akan mendeteksi pelanggaran batas aturan dan langsung melempar (throw) InvalidArgumentException. Pada index.php, alur program dicegat oleh blok catch, yang kemudian menampilkan pesan teks error secara rapi ke layar tanpa sempat memasukkan objek cacat tersebut ke dalam sistem memori / Session.

5. Jawaban Tugas Praktikum

Tugas 1 — Analisis access modifier (individu)

Buat class Produk dengan properti public nama, protected kategori, dan private harga. Uji akses ketiganya dari luar class. Catat hasil dan jelaskan mengapa akses tertentu ditolak. Untuk menguji protected dari class turunan, buat satu subclass ProdukDigital.

Kode program :

```php
<?php
class Produk {
    public string $nama;
    protected string $kategori;
    private int $harga;

    public function __construct(string $nama, string $kategori, int
$harga) {
        $this->nama = $nama;
        $this->kategori = $kategori;
        $this->harga = $harga;
    }
}

class ProdukDigital extends Produk {
    public function tampilkanKategori(): string {
        return "Kategori produk ini adalah: " . $this->kategori;
    }

    // public function tampilkanHarga() {
```

```
    //     return $this->harga;
    // }
}


$produk1 = new ProdukDigital("E-Book Pemrograman PHP", "Buku Digital",
50000);

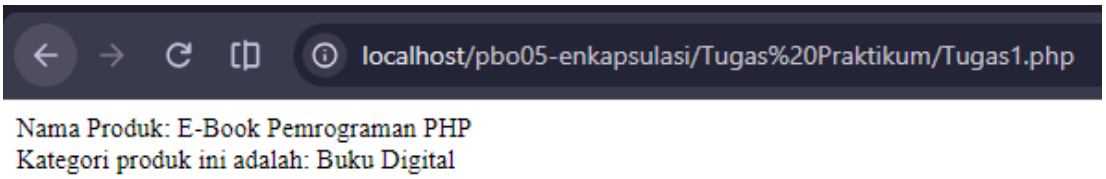
// 1. Menguji Akses PUBLIC dari luar class
echo "Nama Produk: " . $produk1->nama . "<br>";

// 2. Menguji Akses PROTECTED melalui method di subclass
echo $produk1->tampilkanKategori() . "<br>";

// 3. Menguji Akses PROTECTED dari luar class secara langsung
// echo $produk1->kategori;

// 4. Menguji Akses PRIVATE dari luar class secara langsung
// echo $produk1->harga;
```

Hasil pengujian :



Penjelasan :

Berdasarkan pengujian properti $nama yang berakses public dapat dibaca dan diubah secara bebas dari luar class, sedangkan akses langsung ke properti $kategori (protected) dan $harga (private) ditolak oleh PHP dan menghasilkan Fatal Error karena mekanisme enkapsulasi. Akses ke $kategori ditolak dari luar class untuk menyembunyikan data internal, tetapi aksesnya diizinkan jika dipanggil melalui method pada class turunannya (ProdukDigital) yang berada dalam hirarki pewarisan (inheritance). Di sisi lain, properti $harga memiliki proteksi paling ketat karena bersifat private, sehingga diisolasi secara penuh dari script luar maupun subclass dan hanya dapat diakses secara resmi melalui gerbang method getter atau setter di dalam Base Class itu sendiri.

Tugas 2 — Getter, setter, dan validasi (individu)

Buat class Buku dengan properti private kodeBuku, judul, stok. Kode buku hanya dapat dibaca; judul wajib diisi; stok tidak boleh negatif. Buat constructor, getter yang

diperlukan, setJudul(), tambahStok(), dan kurangiStok() dengan pemeriksaan stok mencukupi. Uji setidaknya dua input valid dan tiga input tidak valid.

Kode program :

```php
<?php
class Buku {
    private string $kodeBuku;
    private string $judul;
    private int $stok;

    public function __construct(string $kodeBuku, string $judul, int
$stokAwal = 0) {
        $this->kodeBuku = trim($kodeBuku);
        $this->setJudul($judul);

        if ($stokAwal < 0) {
            throw new InvalidArgumentException("Stok awal tidak boleh
negatif.");
        }
        $this->stok = $stokAwal;
    }

    public function getKodeBuku(): string {
        return $this->kodeBuku;
    }

    public function getJudul(): string {
        return $this->judul;
    }

    public function setJudul(string $judul): void {
        $judul = trim($judul);
        if ($judul === "") {
            throw new InvalidArgumentException("Judul buku wajib
diisi.");
        }
        $this->judul = $judul;
    }

    public function getStok(): int {
        return $this->stok;
    }

    public function tambahStok(int $jumlah): void {
        if ($jumlah <= 0) {
            throw new InvalidArgumentException("Jumlah penambahan stok
harus lebih dari 0.");
        }
        $this->stok += $jumlah;
    }

    public function kurangiStok(int $jumlah): void {
```

```
        if ($jumlah <= 0) {
            throw new InvalidArgumentException("Jumlah pengurangan
stok harus lebih dari 0.");
        }
        if ($this->stok - $jumlah < 0) {
            throw new InvalidArgumentException("Stok tidak mencukupi
untuk dikurangi. Stok saat ini: " . $this->stok);
        }
        $this->stok -= $jumlah;
    }
}

echo "<b>--- 2 PENGUJIAN VALID ---</b><br>";

$buku1 = new Buku("B001", "Pemrograman PHP 8", 10);
echo "Valid 1 (Sukses) -> Buku: " . $buku1->getKodeBuku() . " - " .
$buku1->getJudul() . " (Stok awal: " . $buku1->getStok() . ")<br>";

$buku1->tambahStok(5);
$buku1->kurangiStok(3);
echo "Valid 2 (Sukses) -> Stok akhir setelah transaksi normal: " .
$buku1->getStok() . "<br><br>";

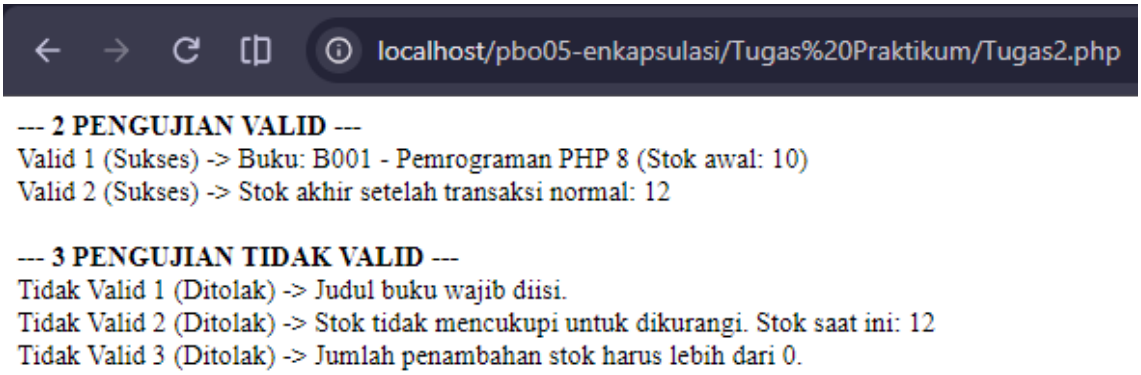

echo "<b>--- 3 PENGUJIAN TIDAK VALID ---</b><br>";

try {
    $buku1->setJudul("");
} catch (Exception $e) {
    echo "Tidak Valid 1 (Ditolak) -> " . $e->getMessage() . "<br>";
}

try {
    $buku1->kurangiStok(100);
} catch (Exception $e) {
    echo "Tidak Valid 2 (Ditolak) -> " . $e->getMessage() . "<br>";
}

try {
    $buku1->tambahStok(-5);
} catch (Exception $e) {
    echo "Tidak Valid 3 (Ditolak) -> " . $e->getMessage() . "<br>";
}
?>
```

Hasil pengujian :



Penjelasan :

Berdasarkan eksekusi kode tugas2.php, enkapsulasi pada class Buku berhasil melindungi data dengan mengubah properti $isbn, $judul, dan $stok menjadi private. Identitas objek seperti ISBN sengaja tidak diberi method setter untuk menjaga sifatnya sebagai data read-only setelah inisialisasi constructor. Selain itu, manipulasi jumlah stok tidak menggunakan setter bebas (setStok()), melainkan dikontrol melalui method berbasis behavior (perilaku) yaitu tambahStok() dan kurangiStok(). Pengujian pembatasan logika terbukti sukses, di mana transaksi pengurangan stok yang melebihi jumlah yang tersedia langsung ditolak oleh sistem dengan melempar InvalidArgumentException, sehingga nilai stok terhindar dari angka negatif dan state objek tetap aman serta konsisten.

Tugas 3 — Refactoring CRUD Mahasiswa (kelompok kecil)

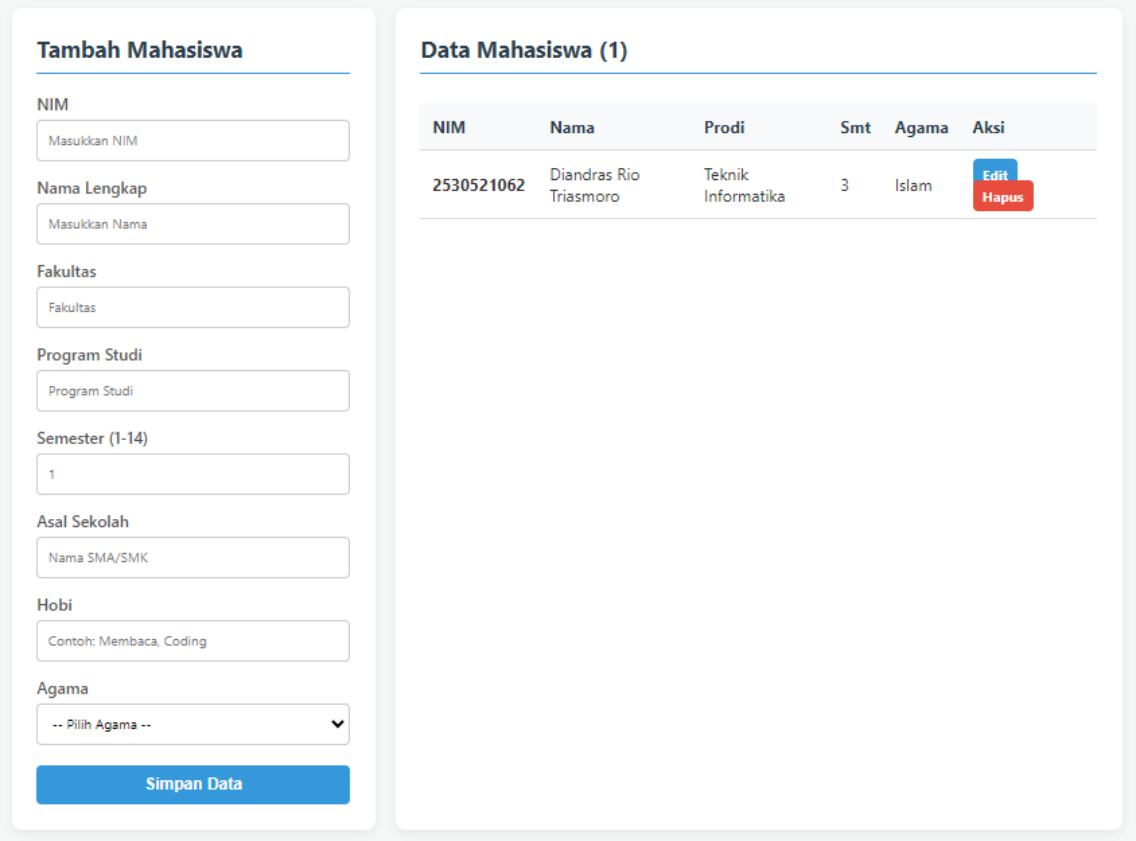
Ubah aplikasi Pertemuan 4 sehingga properti nim, nama, fakultas, prodi, asalSekolah, semester, hobi, dan agama tidak dapat diakses langsung dari index.php. Buat getter untuk tampilan dan setter hanya bagi atribut yang memang boleh diubah. Pertahankan operasi tambah, tampil, edit, hapus. Validasi semester 1–14, NIM wajib dan unik, nama tidak kosong, serta pilihan agama harus termasuk daftar yang diizinkan.

Ketentuan tambahan: NIM ditetapkan tetap setelah object dibuat. Pada fitur Edit, tampilkan NIM sebagai read-only atau non-editable; jangan menyediakan setNim() tanpa rancangan konsistensi key repository. Data tetap menggunakan PHP Session, bukan database.

Kode program :

https://github.com/Diandras-Rio/pbo05-enkapsulasi

Hasil pengujian :



Penjelasan :

Pengembangan aplikasi CRUD Mahasiswa pada Tugas 3 berhasil meminimalisir kesalahan manipulasi data dengan menerapkan enkapsulasi penuh dan pengolahan data berbasis PHP Session. Seluruh properti seperti NIM, Nama, Semester, dan Prodi diisolasi dengan status private, sehingga akses dari antarmuka web wajib melewati method *getter* dan *setter*. Sistem enkapsulasi ini bekerja secara efektif menolak data yang tidak valid melalui pelemparan *InvalidArgumentException* ketika mendeteksi input semester di luar rentang 1–14, kolom wajib yang kosong, maupun NIM yang terdaftar ganda. Selain itu, pembaruan pada kelas *repository* membuat penyimpanan data di dalam Session menjadi lebih aman dan konsisten tanpa merusak data lama yang sudah tersimpan.

Tugas 4 — Refleksi konsep (individu)

- Mengapa private saja belum cukup untuk memastikan object valid?

  Jawaban :

  Hak akses private hanya bertugas mengisolasi properti agar tidak bisa diubah langsung dari luar class. Namun, jika method *setter* yang dibuat tidak dilengkapi dengan kriteria atau aturan pengecekan logika (*validation rules*), data cacat tetap bisa

lolos dan masuk melalui parameter *setter* tersebut. Validitas objek baru terjamin ketika private dikombinasikan dengan logika validasi di dalam *setter*.

- Kapan setter tidak perlu dibuat? Berikan contoh dari class Mahasiswa atau Rekening.
  Jawaban :
  Setter tidak perlu dibuat pada properti yang bersifat read-only atau berfungsi sebagai identitas unik permanen yang tidak boleh diubah setelah objek diinisialisasi melalui constructor.
  Contoh Pada class Mahasiswa, properti $nim hanya diberi getter (getNim()) tanpa adanya setNim(). Hal ini mencegah NIM mahasiswa diubah secara sengaja maupun tidak sengaja setelah objeknya dibuat, karena dalam sistem akademik NIM bersifat tetap dan tidak boleh berganti. Contoh lain adalah properti $isbn pada class Buku.
- Mengapa method tambahStok()/kurangiStok() lebih aman daripada setStok() bebas?
  Jawaban :
  Method berbasis transaksi seperti tambahStok() dan kurangiStok() memaksa perubahan data untuk mengikuti aturan bisnis (perhitungan increment/decrement yang terkontrol). Sebaliknya, setStok() bebas memberi celah bagi pihak luar untuk menimpa nilai stok secara instan dengan angka acak atau bernilai negatif tanpa melalui proses transaksi yang valid.
- Apakah __get()/__set() dari Pertemuan 4 wajib digunakan untuk menerapkan enkapsulasi? Jelaskan.
  Jawaban :
  Tidak wajib. Menggunakan *getter* dan *setter* eksplisit seperti getNama() dan setNama() lebih direkomendasikan karena memudahkan penulisan logika validasi yang berbeda-beda untuk tiap properti, serta menjaga keterbacaan kode (*maintainability*).
- Bagaimana perubahan public menjadi private memengaruhi DataMahasiswa.php dan index.php?
  Jawaban :
  Perubahan tersebut menghilangkan hak akses baca-tulis langsung ke variabel properti. Akibatnya, pada DataMahasiswa.php penentuan *key array* wajib menggunakan method $m->getNim(), dan pada index.php seluruh pemanggilan data

ke form maupun tabel harus diubah menggunakan method *getter* resmi agar tidak memicu *Fatal Error*.

6. **Kesimpulan**

   Enkapsulasi menjaga konsistensi data objek (*object state integrity*) dengan cara melarang pihak luar mengubah variabel properti secara independen atau asal-asalan. Dengan mewajibkan seluruh pengubahan data melewati method *setter* yang dibekali logika validasi, objek akan selalu berada dalam kondisi yang valid (*valid state*) sepanjang siklus hidupnya. Jika ada data tidak rasional seperti semester minus, kolom kosong, atau duplikasi identitas, sistem akan mencegatnya di pintu gerbang *setter* sebelum data tersebut merusak struktur memori atau penyimpanan aplikasi.

7. **Link Github**

   https://github.com/Diandras-Rio/pbo05-enkapsulasi